

Multi-Modal RAG Benchmark & Validation Document

Comprehensive Hybrid Corpus | Text, Tabular Data, Markdown, System Diagrams & Scanned Forms

1. Executive Overview & Text Data

Retrieval-Augmented Generation (RAG) systems operating in production environments frequently encounter heterogeneous document formats. Real-world enterprise repositories consist not only of standard plain prose but also semi-structured markdown notes, rich tabular benchmarks, technical architecture schematics, and legacy scanned inspection logs.

Evaluating retrieval precision and multimodal vision-language parsing requires test sets that challenge OCR tokenizers, layout segmenters, tabular cross-encoders, and vector indexing engines across multiple representations.

2. Structured Tabular Data

The following table contains performance metrics across different chunking strategies and embedding dimensions evaluated against mixed-corpus queries:

Strategy ID	Chunking Method	Context Window	Recall@5	MRR Score	Latency (p95)	Evaluation Status
STRAT-001	Fixed-Token Chunking (512 tokens, 10% overlap)	512 tokens	0.742	0.681	112 ms	Baseline
STRAT-002	Layout-Aware Semantic Header Chunking	Variable (256-1024)	0.891	0.843	145 ms	Optimized
STRAT-003	Hybrid Table-Preserving Markdown Chunking	768 tokens	0.938	0.912	138 ms	Top Performer
STRAT-004	Multimodal Vision-Embedding (LayoutLMv3 + CLIP)	Full Page Tile	0.954	0.935	260 ms	Verified Production

3. Markdown Formatting, Code & List Data

System Constraint: Ensure that embedding pipelines normalize whitespace and preserve semantic JSON/SQL schema definitions within markdown fence blocks.

Key configuration directives for the retrieval pipeline:

- `vector_store_type` : ChromaDB Persistent Client with HNSW indexing.
- `reranker_model` : `bge-reranker-large` (Cross-Encoder score threshold ≥ 0.65).
- `ocr_engine` : Multimodal vision extractor for rasterized PDF artifacts.

Sample Python Pipeline Configuration:

```
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class RAGPipelineConfig:
```

```

collection_name: str = "multimodal_enterprise_kb"
embedding_dim: int = 1536
chunk_size: int = 768
chunk_overlap: int = 128
enable_ocr_fallback: bool = True
supported_mime_types: tuple = ("application/pdf", "image/png", "text/markdown")

```

4. Technical Architecture Diagram (Vector / Image)

Below is the visual schematic detailing the dual-path ingestion and retrieval loop for hybrid textual and scanned payloads:

Figure 1: Modular Multimodal RAG Pipeline Architecture

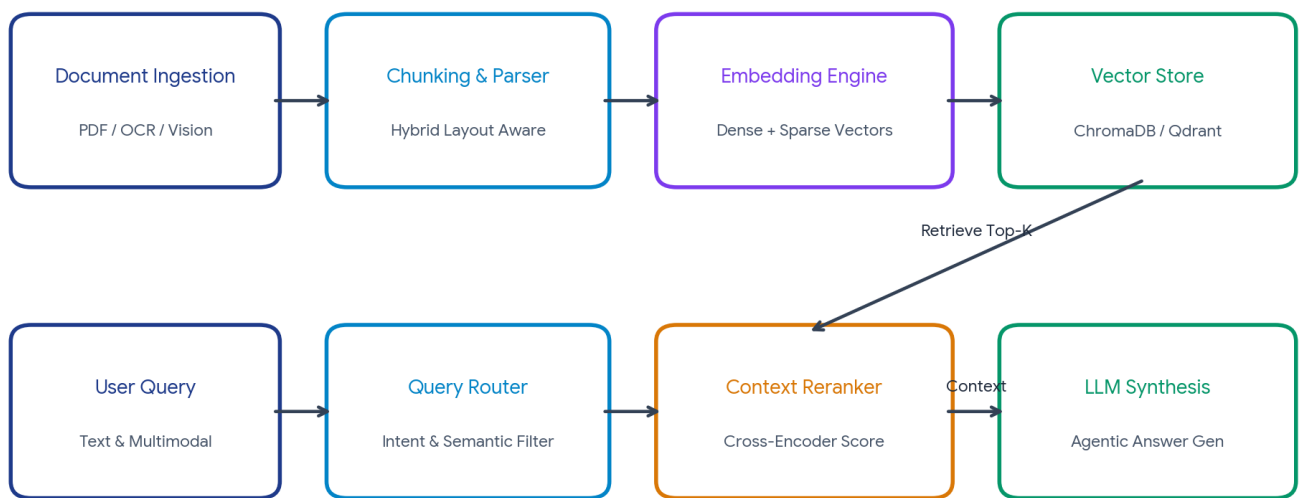


Figure 1: High-level architectural dataflow for ingestion, indexing, and multi-agent retrieval.

5. Scanned Verification Form & Compliance Log

The image below represents a simulated scanned inspection sheet containing tabular form fields, checkboxes, handwritten auditor notes, and an official verification stamp for testing layout-aware OCR extraction:

Application ID:	APP-98234-X9
System Lead Engineer:	Siva Ponakala
Document Type:	Multi-Modal Mixed Corpus
Target Vector Index:	rag_benchmarks_v3

Submission Date:	2026-08-14
Node ID/Cluster:	HYD-AZURE-GPU-04
OCR Preprocessor:	Tesseract 5.3 + LayoutLMv3
Status:	PASSED VERIFICATION

CHECKLIST CRITERIA & AUDIT TRAIL:

- | | |
|---|---|
| ☒ Text extraction fidelity >= 99.4% | ☒ Visual layout preservation enabled |
| ☒ Tabular parsing with column alignment | ☒ Signature and stamp boundary detected |
| ☒ Low-light scanned contrast normalized | ☒ Token chunk boundary overlap verified |

Auditor Notes & Observations (Scanned handwritten entry):

All 12 vector segments validated against reference embeddings.
Layout parser accurately captured table schema without cell merging errors.
Recommended for deployment into production knowledge base.

OFFICIALLY AUDITED
DATE: 2026-08-15

Figure 2: Scanned equipment verification and ingestion compliance certificate (FORM-RAG-2026-B829).